JIST Administrator Guide

Khelo Tech — Self-Hosted Deployment

Satyam Kumar

August 2026

> Created by Satyam Kumar Operating manual for the Khelo Tech self-hosted JIST instance. Written for whoever keeps the server running.

If you only use JIST to track work, read the User Guide instead.

---

1. What we are running

Product	JIST 1.4.0, Community Edition
Instance name	Khelo Tech
Licence	AGPL-3.0, free, unlimited users
Deployment	Docker Compose, prebuilt `khelo-tech/*` images
Compose project	`plane-app`
Install directory	`~/Projects/plane-selfhost`
Current address	`http://localhost:8080` (trial on a MacBook)

The current instance is a **trial on a laptop**. Section 12 covers moving to the production server.

Community Edition limits

Two things worth knowing before you promise them to anyone:

- **No Teamspaces.** That is a paid feature. Team structure is expressed through projects, roles, labels, and modules.
- **No SAML or OIDC.** Google and GitHub OAuth are available; enterprise SSO is not.

Everything in the User Guide — work items, cycles, modules, views, pages, analytics — is in Community Edition.

---

2. The stack

Thirteen containers. Twelve run continuously; the migrator runs once and exits 0, which is correct and not a fault.

Container	Image	Job
`proxy`	`khelo-tech/plane-proxy`	Caddy. Front door, routes everything, terminates TLS
`web`	`khelo-tech/plane-frontend`	Main app at `/`
`admin`	`khelo-tech/plane-admin`	God Mode at `/god-mode`
`space`	`khelo-tech/plane-space`	Public shared views at `/spaces`
`live`	`khelo-tech/plane-live`	Real-time collaborative editing
`api`	`khelo-tech/plane-backend`	Django REST API
`worker`	`khelo-tech/plane-backend`	Celery background jobs
`beat-worker`	`khelo-tech/plane-backend`	Celery scheduled jobs
`migrator`	`khelo-tech/plane-backend`	Database migrations, then exits
`plane-db`	`postgres:15.7-alpine`	Postgres — **all your data**
`plane-redis`	`valkey/valkey:7.2.11-alpine`	Cache and sessions
`plane-mq`	`rabbitmq:3.13.6-management-alpine`	Celery message broker
`plane-minio`	`minio/minio`	File storage — **all attachments**

Routing

Caddy sends traffic by path prefix:

```
/god-mode/*  → admin:3000
/spaces/*    → space:3000
/live/*      → live:3000
/api/*       → api:8000
/auth/*      → api:8000
/static/*    → api:8000
/uploads/*   → plane-minio:9000
/*           → web:3000
```

Volumes

The two that matter are `pgdata` and `uploads`. Lose either and you have lost the deployment.

Volume	Contents	Critical
`plane-app_pgdata`	Postgres. Every work item, comment, user	**Yes**
`plane-app_uploads`	MinIO. Every attachment and avatar	**Yes**
`plane-app_rabbitmq_data`	Queue state	No
`plane-app_redisdata`	Cache	No
`plane-app_proxy_data`	TLS certificates	Regenerable
`plane-app_logs_*`	Service logs	No

---

3. Day-to-day operations

Run everything from the install directory. **The `-p plane-app` flag matters** — the project name does not match the directory name, so omitting it creates a second, empty stack.

```
cd ~/Projects/plane-selfhost
```

Task	Command
Status	`docker compose -p plane-app ps`
Start	`docker compose -p plane-app up -d`
Stop	`docker compose -p plane-app stop`
Restart one service	`docker compose -p plane-app restart api`
Logs, follow	`docker compose -p plane-app logs -f api`
Logs, recent	`docker compose -p plane-app logs --tail=200 api`
Shell into a container	`docker exec -it plane-app-api-1 bash`
Django shell	`docker exec -it plane-app-api-1 python manage.py shell`

Health check in one line:

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/api/instances/
```

`200` means the API, database, and proxy are all working.

Never run `docker compose down -v`. The `-v` deletes volumes, which means the database. There is no undo.

---

4. God Mode

God Mode is the instance admin panel at `/god-mode`. It controls settings for the whole server, above and across all workspaces.

Reach it from your avatar menu, or go to the URL directly.

Sections:

Section	Controls
General	Instance name, telemetry
Email	SMTP settings
Authentication	Sign-up, password login, magic link, OAuth
Artificial Intelligence	OpenAI key for AI features
Images	Unsplash key for cover images
Workspaces	Every workspace on the instance

Instance admin vs workspace admin

Two different things, easily confused:

- **Instance admin** — owns the server. Configures God Mode. Created once at first boot.
- **Workspace admin** — owns a workspace. Manages members and projects inside it. Has no God Mode access.

One person can be both. They are separate grants.

---

5. Authentication — how we have it configured

Setting	Value	Why
`ENABLE_SIGNUP`	`0`	Invite-only. Nobody self-registers
`ENABLE_EMAIL_PASSWORD`	`1`	The only way in
`ENABLE_MAGIC_LINK_LOGIN`	`0`	Needs SMTP, which we do not have
`DISABLE_WORKSPACE_CREATION`	`1`	Only instance admins create workspaces

Workspace creation is restricted on purpose. Work items can only be linked and moved within a single workspace, so a team spinning up their own would cut themselves off from everyone else. That is painful to undo later.

Changing these settings — read this before you try

There are two places these values live, and they behave differently.

`plane.env` **only seeds the database on an instance's very first boot.** The bootstrap command `configure_instance.py` uses `get_or_create`, so on an instance that has already started, editing `plane.env` and restarting does *nothing*. The existing database row wins.

So:

- **On a running instance** → change it in **God Mode** → **Authentication**.
- **On a fresh install** → set it in `plane.env` before the first `up`.

Our `plane.env` carries the correct values so the production server gets them right from the start.

If you must change a value on a running instance without the UI:

```
docker exec plane-app-api-1 python manage.py shell -c "
from plane.license.models import InstanceConfiguration as C
o = C.objects.get(key='ENABLE_SIGNUP')
o.value = '0'
o.save()
"
docker exec plane-app-api-1 python manage.py clear_cache
```

The `clear_cache` is required. `/api/instances/` is cached, so without it the change will not appear.

---

6. Managing people

Roles

Three, at both workspace and project level:

Role	Can
Admin	Everything, including settings and deleting the project
Member	Create and edit work items, cycles, modules, pages
Guest	Restricted view. For contractors and outside collaborators

Give people Member by default. Admin should be a short list — anyone with it can delete a project and everything in it.

Adding someone — the manual route

We have no SMTP, so invitation emails cannot be sent. Until that changes, adding a person takes two steps:

1. **Workspace Settings** → **Members** → **Add Member**, enter their work email, choose a role.
2. Tell them their address and password **through a channel you trust** — in person, or a password manager. Not a group chat.

Then have them change it at Settings → Profile on first sign-in.

Section 11 covers configuring SMTP, which replaces this with a normal invite link and makes password resets self-service. Worth doing before you onboard fifty people.

Resetting a password

Without email, the "Forgot password" flow cannot deliver anything. Reset it yourself:

```
docker exec -it plane-app-api-1 python manage.py shell -c "
from plane.db.models import User
u = User.objects.get(email='person@khelotech.com')
u.set_password('a-temporary-password')
u.save()
print('reset:', u.email)
"
```

Tell them to change it immediately after signing in.

Removing someone

Remove them from **Workspace Settings** → **Members**. Their work items, comments, and history stay — authorship is preserved, access is revoked. That is what you want when someone leaves.

---

7. Structuring Khelo Tech

Recommended shape for Product, Tech, and Testing:

One workspace: `Khelo Tech`**.** Not one per team.

Projects by product or service, not by team. A project called `Mobile App` that all three teams work in beats three projects called Product, Tech, and Testing. Teams collaborate on the same work item rather than copying it between projects and letting the copies drift.

Express the teams through:

- **Labels** — `product`, `backend`, `frontend`, `qa`
- **States** — add "Ready for QA" and "In Review" so handoffs are visible on the board

- **Modules** — group by feature across teams
- **Saved views** — one per team, filtered to their slice, shared with the project

Where separate projects do make sense: genuinely separate work with its own backlog and cadence. An internal tools project, or infrastructure work.

Roles: team leads as project Admin, everyone else Member, contractors Guest.

---

8. Backups

Nothing is backed up automatically. Set this up before the team relies on JIST.

What to back up

`plane-app_pgdata` and `plane-app_uploads`. Postgres alone is not enough — you would restore every work item with every attachment broken.

Database dump

```
docker exec plane-app-plane-db-1 pg_dump -U plane -d plane -Fc \
  > ~/plane-backups/plane-db-$(date +%Y%m%d-%H%M).dump
```

Attachments

```
docker run --rm \
  -v plane-app_uploads:/data:ro \
  -v ~/plane-backups:/backup \
  alpine tar czf /backup/uploads-$(date +%Y%m%d-%H%M).tar.gz -C /data .
```

Daily, via cron

```
0 2 * * * cd ~/Projects/plane-selfhost && \
  docker exec plane-app-plane-db-1 pg_dump -U plane -d plane -Fc > ~/plane-
        backups/db-$(date +\%Y\%m\%d).dump && \
  docker run --rm -v plane-app_uploads:/data:ro -v ~/plane-backups:/backup \
    alpine tar czf /backup/uploads-$(date +\%Y\%m\%d).tar.gz -C /data .
```

Restoring the database

```
docker compose -p plane-app stop api worker beat-worker
cat backup.dump | docker exec -i plane-app-plane-db-1 pg_restore -U plane -d plane --clean
docker compose -p plane-app start api worker beat-worker
```

An untested backup is not a backup. Restore one to a throwaway stack this month, and put a reminder in the calendar to do it again in six.

Keep copies off the server. A backup on the same disk as the database does not survive the disk failing.

---

9. Upgrading

JIST ships releases regularly. The current version and the latest available are both shown at `/api/instances/`.

```
cd ~/Projects/plane-selfhost

# 1. Back up first. Always.
docker exec plane-app-plane-db-1 pg_dump -U plane -d plane -Fc > ~/plane-backups/pre-
        upgrade.dump

# 2. Stop
docker compose -p plane-app stop

# 3. Pull new images
docker compose -p plane-app pull

# 4. Start — the migrator applies schema changes automatically
docker compose -p plane-app up -d

# 5. Watch the migrator finish
docker compose -p plane-app logs -f migrator
```

Read the release notes before upgrading, especially across a major version. Upgrade the trial first and confirm it comes up clean.

The official installer

JIST publishes a `setup.sh` with a menu for Install, Start, Stop, Restart, Upgrade, View Logs, and Backup. Our stack was started with raw `docker compose` and does not have it. Worth adding on the production server:

```
curl -fsSL -o setup.sh https://github.com/makeplane/plane/releases/latest/download/setup.sh
chmod +x setup.sh
```

Note it expects its own directory layout, so try it on the new server rather than retrofitting it here.

---

10. Troubleshooting

Check the obvious first

```
docker compose -p plane-app ps
docker compose -p plane-app logs --tail=100 api
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/api/instances/
```

Symptoms

Site will not load. Check `proxy` is up and the port is not taken by something else: `lsof -nP -iTCP:8080 -sTCP:LISTEN`.

API returns 500. Read `logs api`. Usually the database is unreachable or a migration did not complete.

Migrator keeps restarting. Read `logs migrator`. Normally Postgres was not ready in time; restarting the migrator alone often clears it.

Uploads fail. Check `plane-minio` is up and the file is under `FILE_SIZE_LIMIT` (5 MB by default). Raise it in `plane.env` and restart the proxy and API.

Real-time editing not syncing. Check the `live` container and that `/live/*` reaches it through Caddy.

Notifications and emails not arriving. Expected — no SMTP configured. See section 11.

Setting changed in `plane.env` had no effect. Almost certainly the `get_or_create` behaviour in section 5. Change it in God Mode instead.

Disk

MinIO grows with every attachment. Watch it:

```
docker system df -v | grep plane-app
df -h
```

A full disk stops Postgres from writing, which takes the whole instance down.

---

11. Configuring SMTP

The single highest-value improvement available. It gives you emailed invites, self-service password resets, and notification emails — and removes the manual account handling in section 6.

Set in **God Mode** → **Email**, or in `plane.env` before a first boot:

```
EMAIL_HOST=smtp.your-provider.com
EMAIL_PORT=587
EMAIL_HOST_USER=plane@khelotech.com
EMAIL_HOST_PASSWORD=your-app-password
EMAIL_FROM=JIST <plane@khelotech.com>
EMAIL_USE_TLS=1
EMAIL_USE_SSL=0
```

Send a test from God Mode after saving. Once email works, consider enabling magic-link login — it removes passwords from the equation entirely.

Google Workspace and Microsoft 365 both work with an app password. Amazon SES is a good choice if you are already on AWS.

---

12. Moving to the production server

The current instance is a laptop trial. For 50+ users across three teams:

Hardware

Resource	Minimum	Recommended
vCPU	4	**8**
RAM	8 GB	**16 GB**
Disk	100 GB SSD	**250 GB SSD**
OS	—	Ubuntu 22.04 or 24.04 LTS

Attachments accumulate. Size the disk for a year of them, and monitor it.

Configuration changes

```
# Currently 1 — far too low for 50 users
GUNICORN_WORKERS=8

# Real address, not localhost
APP_DOMAIN=plane.khelotech.com
WEB_URL=https://plane.khelotech.com
CORS_ALLOWED_ORIGINS=https://plane.khelotech.com

# Standard ports
LISTEN_HTTP_PORT=80
LISTEN_HTTPS_PORT=443

# TLS — Caddy obtains and renews the certificate automatically
SITE_ADDRESS=plane.khelotech.com
CERT_EMAIL=admin@khelotech.com

# Do not leave this at the default
TRUSTED_PROXIES=172.16.0.0/12
```

Generate fresh secrets for production. Do not reuse the trial's `SECRET_KEY`, `LIVE_SERVER_SECRET_KEY`, or database passwords:

```
openssl rand -hex 32
```

On `TRUSTED_PROXIES`

The default is `0.0.0.0/0`, which tells Caddy to trust `X-Forwarded-For` from anyone. Any client can then claim any IP, which corrupts rate limiting and audit logs. Set it to your Docker network range.

Checklist

1. Provision the VM, install Docker
2. Point DNS at it — internal or public depending on your access model
3. Copy `docker-compose.yml` and `plane.env`, then edit as above
4. Generate new secrets
5. Configure SMTP before onboarding anyone
6. First boot, create the instance admin at `/god-mode`
7. Verify auth settings took effect from `plane.env`
8. Set up the backup cron and **test a restore**
9. Create the workspace and projects
10. Onboard one team first, then the rest

Access model

You chose internal-IP access. Two things follow:

- **No TLS on a bare IP.** Certificates are issued for names, not addresses. Traffic including passwords crosses your network in plaintext. Acceptable on a trusted LAN, not over the internet.
- **Better:** give it an internal DNS name and a certificate, or put it behind the VPN. Either gets you HTTPS without exposing it publicly.

---

13. Quick reference

```
cd ~/Projects/plane-selfhost

docker compose -p plane-app ps                      # status
docker compose -p plane-app up -d                   # start
docker compose -p plane-app stop                    # stop
docker compose -p plane-app restart api             # restart one
docker compose -p plane-app logs -f api             # follow logs
docker compose -p plane-app pull                    # fetch updates

docker exec -it plane-app-api-1 bash                # shell
docker exec -it plane-app-api-1 python manage.py shell
docker exec plane-app-api-1 python manage.py clear_cache

curl -s http://localhost:8080/api/instances/ | python3 -m json.tool
```

Address	What
`/`	Main app
`/god-mode`	Instance admin
`/spaces`	Public shared views
`/api/instances/`	Health and config, no auth needed

Never run `docker compose down -v`. It deletes the volumes.

JIST 1.4.0 Community Edition · Khelo Tech · August 2026